

FDS 复习—排序算法

6 月 29 日冲刺

1 排序算法总览

算法	平均	最坏	空间	稳定	比较
插入排序	$O(N^2)$	$O(N^2)$	$O(1)$	稳定	是
希尔排序	$O(N^{3/2})$	$O(N^2)$	$O(1)$	不稳	是
堆排序	$O(N \log N)$	$O(N \log N)$	$O(1)$	不稳	是
归并排序	$O(N \log N)$	$O(N \log N)$	$O(N)$	稳定	是
快速排序	$O(N \log N)$	$O(N^2)$	$O(\log N)$	不稳	是
桶排序	$O(M + N)$	—	$O(M)$	稳定	否
基数排序	$O(P(N + B))$	—	$O(N + B)$	稳定	否

考点

比较排序下界：任意基于比较的排序算法，最坏情况下至少需要 $\Omega(N \log N)$ 次比较。
桶排和基数排不基于比较，因此能突破 $O(N \log N)$ 下界。

2 插入排序 (Insertion Sort)

思想：像整理扑克牌，每次从后面取一张插入前面已排好的位置。

```
1 void insertionSort(int A[], int N) {
2     int i, j, tmp;
3     for (i = 1; i < N; i++) {
4         tmp = A[i];
5         for (j = i; j > 0 && A[j-1] > tmp; j--)
6             A[j] = A[j-1];           // 比tmp大的右移
7         A[j] = tmp;                  // 插入到正确位置
8     }
9 }
```

复杂度：

- 最好（已有序）： $O(N)$ — 内层循环一次不进
- 最坏（逆序）： $O(N^2)$

- 平均逆序数： $N(N-1)/4$

常见错误

- 第一次写成了 `tmp = A[i]` 后直接从头找位置，导致不比较大小
- 交换写成 `A[j]=A[j-1]` 忘了 `A[j-1]=A[j]` ——应该只是右移不是交换

3 希尔排序 (Shellsort)

思想：不等距的插入排序。先用大间隔跳着排，逐步缩小间隔，最后间隔 1 = 普通插入排序。

```

1 void shellSort(int A[], int N) {
2     int gap, i, j, tmp;
3     for (gap = N/2; gap > 0; gap /= 2) {
4         for (i = gap; i < N; i++) {
5             tmp = A[i];
6             for (j = i; j >= gap && A[j-gap] > tmp; j -= gap)
7                 A[j] = A[j-gap];
8             A[j] = tmp;
9         }
10    }
11 }
```

和插入排序的区别（只改 3 处）：

1. 外套一层 `for(gap=N/2; gap>0; gap/=2)`
2. `j-1` 变成 `j-gap`
3. `j--` 变成 `j-=gap`

常见错误

- `while(gap!=1)` 吞了 `gap=1` 这轮，最后一轮常规插入没跑
- `i += gap` 只处理了一组，其他组的全漏了
- 内层 `for` 里声明的 `j` 在外面 `A[j]=tmp` 用不了

Hibbard 序列： $h_k = 2^k - 1$ ，最坏 $\Theta(N^{3/2})$ 。

4 堆排序 (Heapsort)

思想：原地最大堆 + 反复取堆顶放到末尾。

- 索引从 0 开始! $\text{left} = 2i+1$, $\text{right} = 2i+2$, $\text{parent} = (i-1)/2$
- 用**最大堆**: 根是最大的, 每次把根和末尾交换后下滤

```

1 void percdDown(int A[], int i, int N) {
2     int child, tmp = A[i];
3     while (2*i + 1 < N) {           // 有左孩子
4         child = 2*i + 1;
5         if (child+1 < N && A[child+1] > A[child]) // 选较大孩子
6             child++;
7         if (tmp >= A[child]) break; // 满足堆序
8         A[i] = A[child];           // 孩子上移
9         i = child;
10    }
11    A[i] = tmp;                     // 放到最终位置
12 }
13
14 void heapSort(int A[], int N) {
15     int i;
16     for (i = N/2 - 1; i >= 0; i--) // BuildMaxHeap: O(N)
17         percdDown(A, i, N);
18     for (i = N - 1; i > 0; i--) { // 排序: O(N log N)
19         int tmp = A[0]; A[0] = A[i]; A[i] = tmp;
20         percdDown(A, 0, i);
21     }
22 }

```

常见错误

- 忘了堆从 0 开始, 孩子/父公式写成 $2*i$ 和 $i/2$
- $j > 1$ 漏了根, 应该是 $j \geq 1$ (ADT 版) 或 $j \geq 0$ (原地版)
- $h \rightarrow \text{size}$ 和 $A[\text{child}]$ 语法混用

复杂度: BuildHeap $O(N)$ + N 次 DeleteMax $O(N \log N) = O(N \log N)$

5 归并排序 (Mergesort)

思想: 分治法。拆成两半递归排, 再合并两个有序数组。

```

1 void merge(int A[], int left, int mid, int right) {
2     int i = left, j = mid + 1, k = 0;

```

```

3     int n = right - left + 1;
4     int* tmp = (int*)malloc(n * sizeof(int));
5
6     while (i <= mid && j <= right) {
7         if (A[i] <= A[j]) tmp[k++] = A[i++];
8         else tmp[k++] = A[j++];
9     }
10    while (i <= mid) tmp[k++] = A[i++];
11    while (j <= right) tmp[k++] = A[j++];
12    for (int p = 0; p < n; p++)
13        A[left + p] = tmp[p];
14    free(tmp);
15 }
16
17 void mergeSort(int A[], int left, int right) {
18     if (left >= right) return;
19     int mid = (left + right) / 2;
20     mergeSort(A, left, mid);
21     mergeSort(A, mid + 1, right);
22     merge(A, left, mid, right);
23 }

```

常见错误

- tmp 的 k 从 left 开始 (应该是从 0)，拷回时搞混
- 忘了 free(tmp)
- n 声明在 malloc 后面，编译错误

6 快速排序 (Quicksort)

思想：选 pivot，小的放左，大的放右，递归。

```

1 int partition(int A[], int left, int right) {
2     int pivot = A[right];           // 选最右边为pivot
3     int i = left;                   // i: 小元素边界
4
5     for (int j = left; j < right; j++) {
6         if (A[j] < pivot) {
7             int tmp = A[i]; A[i] = A[j]; A[j] = tmp;
8             i++;

```

```

9         }
10    }
11    int tmp = A[i]; A[i] = A[right]; A[right] = tmp;
12    return i; // pivot 最终位置
13 }
14
15 void quickSort(int A[], int left, int right) {
16     if (left >= right) return;
17     int p = partition(A, left, right);
18     quickSort(A, left, p - 1);
19     quickSort(A, p + 1, right);
20 }

```

手算示例： $A = [5, 2, 6, 1, 4, 9, 3, 7]$, pivot=7

$j=0: 5 < 7 \rightarrow \text{swap } 5\ 5 \quad i=1 \quad [5, 2, 6, 1, 4, 9, 3, 7]$
 $j=1: 2 < 7 \rightarrow \text{swap } 2\ 2 \quad i=2 \quad [5, 2, 6, 1, 4, 9, 3, 7]$
 $j=2: 6 < 7 \rightarrow \text{swap } 6\ 6 \quad i=3 \quad [5, 2, 6, 1, 4, 9, 3, 7]$
 $j=3: 1 < 7 \rightarrow \text{swap } 1\ 1 \quad i=4 \quad [5, 2, 6, 1, 4, 9, 3, 7]$
 $j=4: 4 < 7 \rightarrow \text{swap } 4\ 4 \quad i=5 \quad [5, 2, 6, 1, 4, 9, 3, 7]$
 $j=5: 9 < 7? \text{ NO} \quad i=5 \quad [5, 2, 6, 1, 4, 9, 3, 7]$
 $j=6: 3 < 7 \rightarrow \text{swap } 9\ 3 \quad i=6 \quad [5, 2, 6, 1, 4, 3, 9, 7]$
 最后 $\text{swap}(7, 9) \rightarrow [5, 2, 6, 1, 4, 3, 7, 9]$
 pivot=7在位置6, 左全<7, 右全>7

优化： Median-of-Three —取 left/mid/right 三元素的中值作为 pivot。

考点

最坏情况：每次 pivot 选到最小或最大 $\rightarrow O(N^2)$

平均： $O(N \log N)$

7 基数排序 (Radix Sort —LSD)

思想：不比较，按每一位丢桶里排序。从最低位 (LSD) 到最高位 (MSD)。

示例： $[64, 8, 216, 512, 27, 729, 0, 1, 343, 125]$

Pass1(个位): 0: [0] 1: [1] 2: [512] 3: [343] 4: [64] 5: [125] 6: [216] 7: [27] 8: [8] 9: [729]
 $\rightarrow 0, 1, 512, 343, 64, 125, 216, 27, 8, 729$

Pass2(十位): 0: [0, 1, 8] 1: [512, 216] 2: [125, 27, 729] 3: [343] 6: [64]
 $\rightarrow 0, 1, 8, 512, 216, 125, 27, 729, 343, 64$

Pass3(百位): 0:[0,1,8,27,64] 1:[125] 2:[216] 3:[343] 5:[512] 7:[729]
→ 0,1,8,27,64,125,216,343,512,729

复杂度: $T = O(P \cdot (N + B))$, P = 位数, B = 基数。

8 表排序 (Table Sort)

动机: 排序大结构体时, 移动代价高。只排序索引表。

原始: $A = [f, d, a, b, e, c]$

$T: [0, 1, 2, 3, 4, 5]$

排序后: $T = [2, 3, 5, 1, 4, 0]$

$T[0]=2 \rightarrow A[2]=a$ (最小)

物理重排: 利用置换环, 最坏 $\lfloor 3N/2 \rfloor$ 次移动。

考点

考点:

- 稳定 vs 不稳定 (考试必问!)
- 各算法的最坏/平均/空间复杂度
- 为什么桶排/基数排能突破 $O(N \log N)$ ——不基于比较
- 决策树 → 比较排序下界 $\Omega(N \log N)$
- 快排 pivot 选不好 → $O(N^2)$